

Size Lowerbounds for Deep Operator Networks

Anirbit

University of Manchester

Reference

This talk is based on work published in Transactions on Machine Learning Research (TMLR) in Feb, 2024.

This paper is co-authored with,

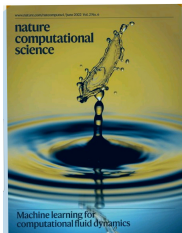
**Amartya Roy, PhD student in IIT-Delhi
(then working @ Bosch, India)**

Table of Contents

1 Introduction

2 The Statement of Our Results

A Resurgence of the AI Driven Ways to Solve Differential Equations



MIT Massachusetts Institute of Technology

Education Research Innovation Admissions + Aid Campus Life

MIT News

ON CAMPUS AND AROUND THE WORLD

[SUBSCRIBE](#) [BROWSE](#)

Technique could efficiently solve partial differential equations for numerous applications

MIT researchers propose "PEDS" method for developing models of complex physical systems in mechanics, optics, thermal transport, fluid dynamics, physical chemistry, climate, and more.

Sandi Miller | Department of Mathematics
January 8, 2024

Quanta Magazine

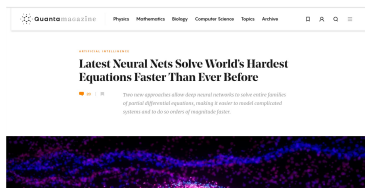
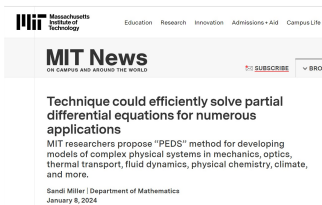
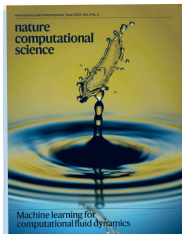
Physics Mathematics Biology Computer Science Topics Archive

ARTIFICIAL INTELLIGENCE

Latest Neural Nets Solve World's Hardest Equations Faster Than Ever Before

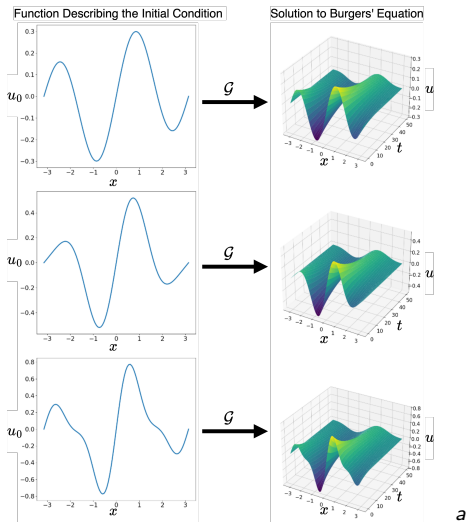
Two new approaches allow deep neural networks to solve entire families of partial differential equations, making it easier to model complicated systems and to do so orders of magnitude faster.

A Resurgence of the AI Driven Ways to Solve Differential Equations



The ability of “Machine Learning” methods to solve differential equations heralds a new era in a plethora of classical fields. And in this talk, we shall study *possibly* one of the most powerful neural methods to solve differential equations - A **DeepONet**

Why Do We Need to Learn Operators?

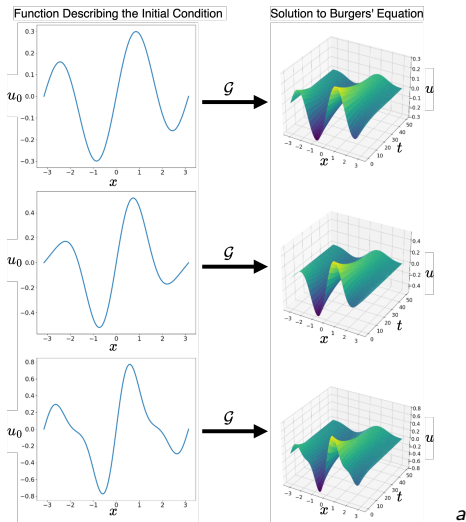


Consider the Burgers' PDE $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$, subject to the periodic boundary condition, $u(x - \pi, t) = u(x + \pi, t)$ and initial conditions $u(x, 0) = u_0(x)$.

Solutions to such a PDE for varying initial conditions can be said to define an operator $\mathcal{G}(u_0) = u$ that maps u_0 to the corresponding solution u .

^a(figure from my PhD student Dibyakanti)

Why Do We Need to Learn Operators?



Consider the Burgers' PDE $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$, subject to the periodic boundary condition, $u(x - \pi, t) = u(x + \pi, t)$ and initial conditions $u(x, 0) = u_0(x)$.

Solutions to such a PDE for varying initial conditions can be said to define an operator $\mathcal{G}(u_0) = u$ that maps u_0 to the corresponding solution u .

Can Data-Driven Algorithms Learn Such Infinite Dimensional Operators?

^a(figure from my PhD student Dibyakanti)

Defining DeepONets: A Schematic Overview.

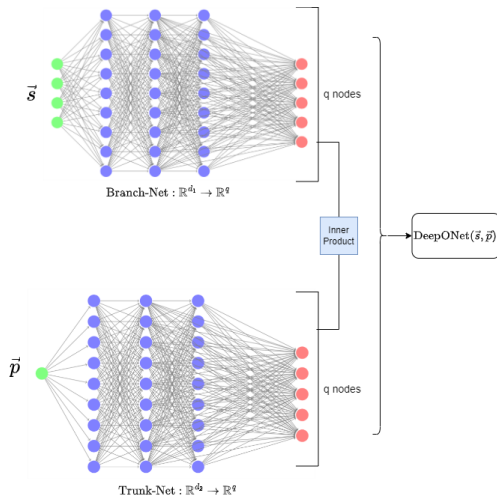


Figure 1: A Sketch of the DeepONet Architecture

How to Use a DeepONet?

Recall our example of the Burgers' PDE, $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$ being solved at different initial conditions $u_0(x)$.

How to Use a DeepONet?

Recall our example of the Burgers' PDE, $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$ being solved at different initial conditions $u_0(x)$. Here the training/test data sets would be collections of 3-tuples of the form,

$$(\vec{u}_0, \vec{z} = (x, t), u(\vec{z}))$$

where $\vec{z} \in \mathbb{R}^2$ is the space-time location where we assume (noisy) knowledge of the true solution u when the initial condition is u_0 — which is given in a discretized form $\vec{u}_0$.

How to Use a DeepONet?

Recall our example of the Burgers' PDE, $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$ being solved at different initial conditions $u_0(x)$. Here the training/test data sets would be collections of 3-tuples of the form,

$$(\vec{u}_0, \vec{z} = (x, t), u(\vec{z}))$$

where $\vec{z} \in \mathbb{R}^2$ is the space-time location where we assume (noisy) knowledge of the true solution u when the initial condition is u_0 — which is given in a discretized form $\vec{u}_0$.

Given m such training data samples, the ℓ_2 — empirical loss of a DeepONet, $\tilde{\mathcal{G}}$ attempting to solve the Burgers' PDE would be,

How to Use a DeepONet?

Recall our example of the Burgers' PDE, $\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} = \kappa \frac{\partial^2 u}{\partial x^2}$ being solved at different initial conditions $u_0(x)$. Here the training/test data sets would be collections of 3-tuples of the form,

$$(\vec{u}_0, \vec{z} = (x, t), u(\vec{z}))$$

where $\vec{z} \in \mathbb{R}^2$ is the space-time location where we assume (noisy) knowledge of the true solution u when the initial condition is u_0 — which is given in a discretized form $\vec{u}_0$.

Given m such training data samples, the ℓ_2 — empirical loss of a DeepONet, $\tilde{\mathcal{G}}$ attempting to solve the Burgers' PDE would be,

$$\hat{\mathcal{L}}(\tilde{\mathcal{G}}) := \frac{1}{2m} \sum_{i=1}^m (u(\vec{z}_i) - \tilde{\mathcal{G}}(\vec{u}_{i0}, \vec{z}_i))^2$$

No Need To Know The PDE!

Imagine the cost of delays in solving the wave PDE in a tsunami warning system! Such mission critical use-cases neither have exact PDE descriptions nor can they afford delays.

No Need To Know The PDE!

Imagine the cost of delays in solving the wave PDE in a tsunami warning system! Such mission critical use-cases neither have exact PDE descriptions nor can they afford delays.

But this loss function to be minimized **only needs a sufficient number of valid input-output pairs to be given beforehand for training and not a knowledge of the underlying PDE!**

Given observation data a DeepONet can still find the right map — “instantaneously” at inference — wholly unlike how finite element type methods which could take hours on such PDEs!

No Need To Know The PDE!

Imagine the cost of delays in solving the wave PDE in a tsunami warning system! Such mission critical use-cases neither have exact PDE descriptions nor can they afford delays.

But this loss function to be minimized **only needs a sufficient number of valid input-output pairs to be given beforehand for training and not a knowledge of the underlying PDE!**,

Given observation data a DeepONet can still find the right map — “instantaneously” at inference — wholly unlike how finite element type methods which could take hours on such PDEs!

No Need To Know The PDE!

Imagine the cost of delays in solving the wave PDE in a tsunami warning system! Such mission critical use-cases neither have exact PDE descriptions nor can they afford delays.

But this loss function to be minimized **only needs a sufficient number of valid input-output pairs to be given beforehand for training and not a knowledge of the underlying PDE!**,

Given observation data a DeepONet can still find the right map — “instantaneously” at inference — wholly unlike how finite element type methods which could take hours on such PDEs!

This is a distinguishing feature and promise of “Operator Learning”.

How to Use a DeepONet?

Suppose $U \subset \mathbb{R}^n$ is the compact domain on which we seek the PDE solutions and $D \subset \mathbb{R}^d$ is the domain of the forcing functions. Then we can imagine 2 associated operators,

- A DeepONet, $\mathcal{N} : C(D) \rightarrow L^2(U)$ i.e $\mathcal{N}(f)(\mathbf{x}_T) := \text{DON}(\mathbf{x}_B(f), \mathbf{x}_T)$.
- A solution operator $\mathcal{G} : C(D) \rightarrow L^2(U)$ which maps forcing functions to PDE solutions.

How to Use a DeepONet?

Suppose $U \subset \mathbb{R}^n$ is the compact domain on which we seek the PDE solutions and $D \subset \mathbb{R}^d$ is the domain of the forcing functions. Then we can imagine 2 associated operators,

- A DeepONet, $\mathcal{N} : C(D) \rightarrow L^2(U)$ i.e $\mathcal{N}(f)(\mathbf{x}_T) := \text{DON}(\mathbf{x}_B(f), \mathbf{x}_T)$.
- A solution operator $\mathcal{G} : C(D) \rightarrow L^2(U)$ which maps forcing functions to PDE solutions.

Then it has been shown that, if μ is a measure s.t $\mathcal{G} \in L^2(\mu)$, then for every $\epsilon > 0$, there exists an operator network $\mathcal{N} : C(D) \rightarrow L^2(U)$, such that,

$$\|\mathcal{G} - \mathcal{N}\|_{L^2(\mu)} = \left(\int_{C(D)} \|\mathcal{G}(f) - \mathcal{N}(f)\|_{L^2(U)}^2 d\mu(f) \right)^{1/2} < \epsilon.$$

(S. Lanthaler, S. Mishra, G. E. Karniadakis, doi.org/10.1093/imatrm/tnac001 (2022))

How to Use a DeepONet?

Suppose $U \subset \mathbb{R}^n$ is the compact domain on which we seek the PDE solutions and $D \subset \mathbb{R}^d$ is the domain of the forcing functions. Then we can imagine 2 associated operators,

- A DeepONet, $\mathcal{N} : C(D) \rightarrow L^2(U)$ i.e $\mathcal{N}(f)(\mathbf{x}_T) := \text{DON}(\mathbf{x}_B(f), \mathbf{x}_T)$.
- A solution operator $\mathcal{G} : C(D) \rightarrow L^2(U)$ which maps forcing functions to PDE solutions.

Then it has been shown that, if μ is a measure s.t $\mathcal{G} \in L^2(\mu)$, then for every $\epsilon > 0$, there exists an operator network $\mathcal{N} : C(D) \rightarrow L^2(U)$, such that,

$$\|\mathcal{G} - \mathcal{N}\|_{L^2(\mu)} = \left(\int_{C(D)} \|\mathcal{G}(f) - \mathcal{N}(f)\|_{L^2(U)}^2 d\mu(f) \right)^{1/2} < \epsilon.$$

(S. Lanthaler, S. Mishra, G. E. Karniadakis, doi.org/10.1093/imatrm/tnac001 (2022))

The above gives an “**universal approximation theorem for operators**”

How DeepONets Train : An Intuition

Intuition about how a DeepONet works...

If $(\beta_i(\mathbf{x}_B), \tau_i(\mathbf{x}_T))$, $i = 1, \dots, q$ are the output coordinates of the branch and the trunk net then, the quantity

“DeepONet($\mathbf{x}_B, \mathbf{x}_T$) = $\sum_{i=1}^q \beta_i(\mathbf{x}_B) \cdot \tau_i(\mathbf{x}_T$)” can be read as :

the branch network output gives the coefficients with which to linearly combine the trunk network's coordinate functions - which are vectors/functions in the space where we are seeking the PDE solutions.

How DeepONets Train : An Intuition

Intuition about how a DeepONet works...

If $(\beta_i(\mathbf{x}_B), \tau_i(\mathbf{x}_T)), i = 1, \dots, q$ are the output coordinates of the branch and the trunk net then, the quantity

“DeepONet($\mathbf{x}_B, \mathbf{x}_T$) = $\sum_{i=1}^q \beta_i(\mathbf{x}_B) \cdot \tau_i(\mathbf{x}_T$)” can be read as :

the branch network output gives the coefficients with which to linearly combine the trunk network's coordinate functions - which are vectors/functions in the space where we are seeking the PDE solutions.

Hence heuristically, this is joint optimization over 2 objectives:

- (a) Finding q good functions in the solution space
(Learning the trunk)
- & (b) Finding a good vector in the span of the former.
(Learning the branch).

Table of Contents

1 Introduction

2 The Statement of Our Results

A Size Requirement for Deep Operator Nets to be “Good”

Theorem (Informal Statement of Our Main Theorem)

With high probability over sampling n noisy training data, if a class of fixed size bounded output DeepONets has to have a predictor that can achieve empirical training error below a label noise dependent threshold, then necessarily q — the common output dimension of the branch and the trunk — must be lower bounded as $q = \Omega(\sqrt[4]{n})$

Speciality of our Theorem

- This stands as one of the very rare instances where a proof of architectural constraint for training performance for any neural network structure has been established.

Speciality of our Theorem

- This stands as one of the very rare instances where a proof of architectural constraint for training performance for any neural network structure has been established.
- This lower-bound is “universal” i.e it’s independent of the specific Partial Differential Equation that a DeepONet aims to solve.

The Formal Setup for DeepONet Theory

Definition (Training Dataset Property)

$(y_i, (\mathbf{s}_i, \mathbf{p}_i))$ be i.i.d. input-output pair, And $y_i \in [-B, B]$ and we define the conditional random variable $g(\mathbf{s}_i, \mathbf{p}_i) = \mathbb{E}[y \mid (\mathbf{s}_i, \mathbf{p}_i)]$

Definition (Branch Functions & Trunk Functions)

$$\mathcal{B} := \left\{ B_{\mathbf{w}} \mid B_{\mathbf{w}} : \mathbb{R}^{d_1} \rightarrow \mathbb{R}^q, \text{Lip}(B_{\mathbf{w}}) \leq L_B \& \|\mathbf{w}\|_2 \leq W_B \& \|B_{\mathbf{w}}\|_{\infty} \leq \mathcal{C} \right\}$$

$$\mathcal{T} := \left\{ T_{\mathbf{w}} \mid T_{\mathbf{w}} : \mathbb{R}^{d_2} \rightarrow \mathbb{R}^q, \text{Lip}(T_{\mathbf{w}}) \leq L_T \& \|\mathbf{w}\|_2 \leq W_T \& \|T_{\mathbf{w}}\|_{\infty} \leq \mathcal{C} \right\}$$

The bound of $\mathcal{C}$ in the above definitions abstracts out the model of the branch and the trunk functions being nets having a layer of bounded activation functions in their output layer.

The Formal Setup for DeepONet Theory

Definition (**The DeepONet(DON) Class**)

$$\mathcal{H} := \{h_{\mathbf{w}_b, \mathbf{w}_t} = h_{(\mathbf{w}_b, \mathbf{w}_t)} \mid \\ \mathbb{R}^{d_1} \times \mathbb{R}^{d_2} \ni (\mathbf{s}, \mathbf{p}) \mapsto h(\mathbf{s}, \mathbf{p}) := \langle B_{\mathbf{w}_b}(\mathbf{s}), T_{\mathbf{w}_t}(\mathbf{p}) \rangle \in \mathbb{R}, \\ B_{\mathbf{w}_b} \in \mathcal{B} \ \& \ T_{\mathbf{w}_t} \in \mathcal{T}\}$$

The Formal Setup for DeepONet Theory

Definition (The DeepONet(DON) Class)

$$\mathcal{H} := \{h_{\mathbf{w}_b, \mathbf{w}_t} = h_{(\mathbf{w}_b, \mathbf{w}_t)} \mid$$

$$\mathbb{R}^{d_1} \times \mathbb{R}^{d_2} \ni (\mathbf{s}, \mathbf{p}) \mapsto h(\mathbf{s}, \mathbf{p}) := \langle B_{\mathbf{w}_b}(\mathbf{s}), T_{\mathbf{w}_t}(\mathbf{p}) \rangle \in \mathbb{R},$$

$$B_{\mathbf{w}_b} \in \mathcal{B} \text{ \& } T_{\mathbf{w}_t} \in \mathcal{T}\}$$

Note that $\forall \theta > 0$, $\exists$ a “ θ -cover” of this function space $\mathcal{H}_\theta$ such that,

$$\forall h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H}, \exists h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})} \in \mathcal{H}_\theta, \text{ s.t.}$$

$\left\| \mathbf{w}_b - \mathbf{w}_{b, \frac{\theta}{2}} \right\| \leq \frac{\theta}{2}$ and $\left\| \mathbf{w}_t - \mathbf{w}_{t, \frac{\theta}{2}} \right\| \leq \frac{\theta}{2}$ and $\mathbf{w}_{b, \frac{\theta}{2}}$ and $\mathbf{w}_{t, \frac{\theta}{2}}$ being elements of the $\frac{\theta}{2}$ covering space of the set of branch and trunk weights respectively.

The Formal Setup for DeepONet Theory

Definition (Defining J /Lipschitzness in Weights of the Nets)

Given any two valid weight vectors w_1 and w_2 for a “branch function” B we assume to have the following inequality for some fixed $J > 0$,

$$\sup_s \|B_{w_1}(s) - B_{w_2}(s)\|_\infty \leq J \cdot \|w_1 - w_2\|$$

And similarly for the trunk functions.

The Formal Setup for DeepONet Theory

Definition (Defining J /Lipschitzness in Weights of the Nets)

Given any two valid weight vectors w_1 and w_2 for a “branch function” B we assume to have the following inequality for some fixed $J > 0$,

$$\sup_s \|B_{w_1}(s) - B_{w_2}(s)\|_\infty \leq J \cdot \|w_1 - w_2\|$$

And similarly for the trunk functions.

That such a Lipschitzness property holds for nets has been proven many times, including in iconic papers like, [A Universal Law of Robustness via Isoperimetry](#) by Sebastien Bubeck and Mark Sellke — the work which inspires our analysis most closely.

4 Lemmas of The Story

Lemma (1. A Standard Result About Covering Numbers)

For any space X with Euclidean metric, we denote as $N(\theta, X)$ the covering number of it at scale θ . Further recall that d_B and d_T are the total number of parameters in any branch and trunk function in the chosen sets $\mathcal{B}$ and $\mathcal{T}$, respectively – and let their weight spaces be $\mathcal{W}_B$ and $\mathcal{W}_T$.

Let $\mathcal{W}_H = \mathcal{W}_B \times \mathcal{W}_T$ denote the sets of allowed weights of $\mathcal{H}$ — the corresponding DeepONet class. Then the following three bounds hold for any $\theta > 0$,

$$N(\theta, \mathcal{W}_B) \leq \left(\frac{2W_B \sqrt{d_B}}{\theta} \right)^{d_B} \quad N(\theta, \mathcal{W}_T) \leq \left(\frac{2W_T \sqrt{d_T}}{\theta} \right)^{d_T}$$

$$N(\theta, \mathcal{W}_H) \leq N(\theta/2, \mathcal{W}_B) \cdot N(\theta/2, \mathcal{W}_T)$$

4 Lemmas of The Story

Lemma (2. Cover Shifting Lemma)

We recall the definition of $\mathcal{H}$, the DON class, B the label bound & J from weight Lipschitzness of the nets. For any $h \in \mathcal{H}$ and any training data we denote the corresponding empirical risk as,

$\hat{R}(h) := \frac{1}{n} \sum_{i=1}^n (y_i - h(\mathbf{s}_i, \mathbf{p}_i))^2$. Then, $\forall \theta > 0$ we have,

$$\hat{\mathcal{R}}(h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}) \leq \hat{\mathcal{R}}(h_{(\mathbf{w}_b, \mathbf{w}_t)}) + qCJ\theta \cdot (B + 2qC^2)$$

and $\mathbf{w}_{b, \frac{\theta}{2}}$ and $\mathbf{w}_{t, \frac{\theta}{2}}$ be s.t. $\|\mathbf{w}_b - \mathbf{w}_{b, \frac{\theta}{2}}\| \leq \frac{\theta}{2}$ and $\|\mathbf{w}_t - \mathbf{w}_{t, \frac{\theta}{2}}\| \leq \frac{\theta}{2}$.

4 Lemmas of The Story

Lemma (2. Cover Shifting Lemma)

We recall the definition of $\mathcal{H}$, the DON class, B the label bound & J from weight Lipschitzness of the nets. For any $h \in \mathcal{H}$ and any training data we denote the corresponding empirical risk as,

$\hat{R}(h) := \frac{1}{n} \sum_{i=1}^n (y_i - h(\mathbf{s}_i, \mathbf{p}_i))^2$. Then, $\forall \theta > 0$ we have,

$$\hat{\mathcal{R}}(h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}) \leq \hat{\mathcal{R}}(h_{(\mathbf{w}_b, \mathbf{w}_t)}) + qCJ\theta \cdot (B + 2qC^2)$$

and $\mathbf{w}_{b, \frac{\theta}{2}}$ and $\mathbf{w}_{t, \frac{\theta}{2}}$ be s.t. $\|\mathbf{w}_b - \mathbf{w}_{b, \frac{\theta}{2}}\| \leq \frac{\theta}{2}$ and $\|\mathbf{w}_t - \mathbf{w}_{t, \frac{\theta}{2}}\| \leq \frac{\theta}{2}$.

Thus we see that it is quantifiable as to how much is the increment in the empirical risk, when for a given training data a DeepONet is replaced by another with weights within a distance of $\mathcal{O}(\theta)$ from the original - and that this increment is parametric in θ .

4 Lemmas of The Story

Lemma (3. Correlation Bounding Lemma)

$\forall \theta > 0$, and for $z_i := y_i - g(\mathbf{s}_i, \mathbf{p}_i) = y_i - \mathbb{E}[y \mid (\mathbf{s}_i, \mathbf{p}_i)]$,

$$\begin{aligned} & \mathbb{P} \left(\exists h(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}}) \in \mathcal{H}_\theta \mid \frac{1}{n} \sum_{i=1}^n h(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})(\mathbf{s}_i, \mathbf{p}_i) z_i \geq \frac{\theta}{4} \right) \\ & \leq \frac{2^{2(d_B + d_T) + 1}}{\theta(d_B + d_T)} \cdot \left(W_B \sqrt{d_B} \right)^{d_B} \cdot \left(W_T \sqrt{d_T} \right)^{d_T} \cdot \exp \left(-\frac{2n\theta^2}{8^4 \cdot (B \cdot qC^2)^2} \right) \\ & + 2 \exp \left(-\frac{n\theta^2}{8^3 \cdot B^2 \cdot q^2 \cdot C^4} \right) \end{aligned}$$

4 Lemmas of The Story

Lemma (3. Correlation Bounding Lemma)

$\forall \theta > 0$, and for $z_i := y_i - g(\mathbf{s}_i, \mathbf{p}_i) = y_i - \mathbb{E}[y \mid (\mathbf{s}_i, \mathbf{p}_i)]$,

$$\begin{aligned} & \mathbb{P} \left(\exists h(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}}) \in \mathcal{H}_\theta \mid \frac{1}{n} \sum_{i=1}^n h(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})(\mathbf{s}_i, \mathbf{p}_i) z_i \geq \frac{\theta}{4} \right) \\ & \leq \frac{2^{2(d_B + d_T) + 1}}{\theta^{(d_B + d_T)}} \cdot \left(W_B \sqrt{d_B} \right)^{d_B} \cdot \left(W_T \sqrt{d_T} \right)^{d_T} \cdot \exp \left(-\frac{2n\theta^2}{8^4 \cdot (B \cdot qC^2)^2} \right) \\ & + 2 \exp \left(-\frac{n\theta^2}{8^3 \cdot B^2 \cdot q^2 \cdot C^4} \right) \end{aligned}$$

The RHS above shows that the possibility of a predictor in the cover space with high average correlation with label noise, falls exponentially with the #training data.

Proof Sketch for Lemma [3]

- For each data i and each predictor $h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}$, we further define the random variable,

$$Y_{\theta,i} := \left(h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}(\mathbf{s}_i, \mathbf{p}_i) - \mathbb{E}[h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}] \underbrace{(y_i - g(\mathbf{s}_i, \mathbf{p}_i))}_{z_i} \right)$$

- One can show $Y_{\theta,i}$ is a centered and a bounded r.v and then one can prove concentration bounds for the data average of it.
- The lemma follows by union bound on the finite class of functions $\mathcal{H}_\theta$

4 Lemmas of The Story

Lemma (4. Low Correlation To Noise Implies High Error DeepONet)

We continue in the same setup as in the previous lemma and further recall the definition of $\sigma^2 := \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[(y_i - g(\mathbf{s}_i, \mathbf{p}_i))^2 \right]$ and $g(\mathbf{s}, \mathbf{p}) = \mathbb{E}[y \mid (\mathbf{s}, \mathbf{p})]$. Then $\forall \theta > 0$

$$\begin{aligned} & \mathbb{P} \left(\exists h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H} \mid \frac{1}{n} \sum_{i=1}^n (y_i - h_{\mathbf{w}_b, \mathbf{w}_t}(\mathbf{s}_i, \mathbf{p}_i))^2 \leq \sigma^2 - \theta \right) \\ & \leq 2 \exp \left(-\frac{n\theta^2}{288B^2} \right) + \mathbb{P} \left(\exists h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H} \mid \frac{1}{n} \sum_{i=1}^n h(\mathbf{s}_i, \mathbf{p}_i) z_i \geq \frac{\theta}{4} \right) \end{aligned}$$

4 Lemmas of The Story

Lemma (4. Low Correlation To Noise Implies High Error DeepONet)

We continue in the same setup as in the previous lemma and further recall the definition of $\sigma^2 := \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[(y_i - g(\mathbf{s}_i, \mathbf{p}_i))^2 \right]$ and $g(\mathbf{s}, \mathbf{p}) = \mathbb{E}[y \mid (\mathbf{s}, \mathbf{p})]$. Then $\forall \theta > 0$

$$\begin{aligned} & \mathbb{P} \left(\exists h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H} \mid \frac{1}{n} \sum_{i=1}^n (y_i - h_{\mathbf{w}_b, \mathbf{w}_t}(\mathbf{s}_i, \mathbf{p}_i))^2 \leq \sigma^2 - \theta \right) \\ & \leq 2 \exp \left(-\frac{n\theta^2}{288B^2} \right) + \mathbb{P} \left(\exists h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H} \mid \frac{1}{n} \sum_{i=1}^n h(\mathbf{s}_i, \mathbf{p}_i) z_i \geq \frac{\theta}{4} \right) \end{aligned}$$

The above lemma reveals an intimate connection between the empirical error of DeepONets and the correlation of its output with label noise.

Proof Sketch : Step 1

What We Want to Ensure,

$$\forall \delta \in (0, 1), (1 - \delta) \leq \mathbb{P}(\text{low-error-DeepONet-exists})$$

Proof Sketch : Step 1

What We Want to Ensure,

$$\forall \delta \in (0, 1), (1 - \delta) \leq \mathbb{P}(\text{low-error-DeepONet-exists})$$

To Find the Necessary Conditions for the Above, the Strategy is to Find an *Unconditional* Upperbound on this Probability of the Good Event that Concerns Us.

Proof Sketch : Step 1

First, we invoke on $\mathcal{H}_\theta$ the Last Lemma “High Correlation To Noise Implies Low Error DeepONet”

$$\begin{aligned} & \mathbb{P} \left(\exists h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})} \in \mathcal{H}_\theta \mid \frac{1}{n} \sum_{i=1}^n \left(y_i - h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}(\mathbf{s}_i, \mathbf{p}_i) \right)^2 \leq \sigma^2 - \theta \right) \\ & \leq 2 \exp \left(-\frac{n\theta^2}{288 \cdot B^2} \right) \\ & + \mathbb{P} \left(\exists h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})} \in \mathcal{H}_\theta \mid \frac{1}{n} \sum_{i=1}^n h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}(\mathbf{s}_i, \mathbf{p}_i) z_i \geq \frac{\theta}{4} \right) \end{aligned}$$

Step-2

Next, we invoke on the RHS above the “Correlation Bounding Lemma”,

$$\begin{aligned}
 & \mathbb{P} \left(\exists h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})} \in \mathcal{H}_\theta \mid \frac{1}{n} \sum_{i=1}^n \left(y_i - h_{(\mathbf{w}_{b, \frac{\theta}{2}}, \mathbf{w}_{t, \frac{\theta}{2}})}(\mathbf{s}_i, \mathbf{p}_i) \right)^2 \leq \sigma^2 - \theta \right) \\
 & \leq 2 \exp \left(-\frac{n\theta^2}{288 \cdot B^2} \right) + \frac{2^{2(d_B+d_T)+1}}{\theta^{(d_B+d_T)}} \cdot \left(W_B \sqrt{d_B} \right)^{d_B} \cdot \left(W_T \sqrt{d_T} \right)^{d_T} \\
 & \cdot \exp \left(-\frac{2n\theta^2}{8^4 \cdot (B \cdot qC^2)^2} \right) + 2 \exp \left(-\frac{n\theta^2}{8^3 \cdot (B \cdot q \cdot C^2)^2} \right)
 \end{aligned}$$

Step-3

From the “Cover Shifting Lemma” we know how the error of a predictor changes between itself and its closest cover point.

Using that we can derive an upperbound on the probability of a good DeepONet to exist in the function class.

Step-3

From the “Cover Shifting Lemma” we know how the error of a predictor changes between itself and its closest cover point.

Using that we can derive an upperbound on the probability of a good DeepONet to exist in the function class.

The required necessary condition now follows from demanding that this upperbound be above $1 - \delta$, (after a bunch of messy algebra!).

Step-3...Continues

Main Theorem : Lowerbounds for DeepONets Whose Branch and Trunk End in Sigmoid Gates

Let W be a bound on the norms of the weights of the branch and the trunk and s , the total number of trainable parameters.

Then $\forall \delta \in (0, 1)$, and any arbitrary positive constant $\epsilon > 0$ if with probability at least $1 - \delta$ with respect to the sampling of the data $\{(y_i, (\mathbf{s}_i, \mathbf{p}_i)) \mid i = 1, \dots, n\}$, $\exists h_{\mathbf{w}_b, \mathbf{w}_t} \in \mathcal{H}$ s.t,

$$\frac{1}{n} \sum_{i=1}^n (y_i - h_{\mathbf{w}_b, \mathbf{w}_t}(\mathbf{s}_i, \mathbf{p}_i))^2 \leq \sigma^2 - \epsilon (1 + J \cdot (B + 2))$$

Step-3...Continues

Main Theorem : Lowerbounds for DeepONets Whose Branch and Trunk End in Sigmoid Gates

then,

$$q \geq n^{\frac{1}{4}} \cdot \left(\frac{\epsilon^2}{288 \cdot B^2} \cdot \frac{1}{\ln \left(2 + e^{-s \cdot \alpha'} \cdot \left(\frac{4 \cdot \min\{d_B, d_T\}^2}{\epsilon} \cdot W \sqrt{s} \right)^s \right) + \ln \left(\frac{2}{1-\delta} \right)} \right)^{\frac{1}{4}}$$

where $\sigma^2 := \frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[(y_i - g(\mathbf{s}_i, \mathbf{p}_i))^2 \right]$ and $g(\mathbf{s}, \mathbf{p}) = \mathbb{E}[y \mid (\mathbf{s}, \mathbf{p})]$ and if the branch net has α -fraction of the training parameters then $\alpha' = \frac{\alpha}{2} \ln \frac{1}{\alpha} + \frac{1-\alpha}{2} \ln \frac{1}{1-\alpha}$.

Experimental Set-up

In this experiment, we aim to show that at a fixed number of parameters, increasing the output dimension q and the training data as q^2 results in a monotonic decrease in DeepONet training error. We use the advection-diffusion-reaction (ADR) PDE as a test case, given by:

$$\frac{\partial u}{\partial t} = D \frac{\partial^2 u}{\partial x^2} + ku^2 + f(x), \quad x \in [0, 1], t \in [0, 1]$$

(at $D = 0.01$ and $k = 0.01$)

The DeepONet empirical loss being attempted to be minimized is $\hat{\mathcal{L}}_{\text{DeepONet}} := \frac{1}{n} \sum_{i=1}^n (y_i - \mathcal{G}_\theta(f_i)(p_i))^2$, where p_i s are points in the (x, t) space and y_i s are the value of the PDE solution at p_i when the inhomogeneous term is f_i .

Experiments

We trained 10 DeepONet models with varying widths, keeping total size equal. Performance increased monotonically when $\frac{q}{\sqrt{n}}$ was constant but not when $\frac{q}{n^{2/3}}$ was constant, indicating insufficient data scaling.

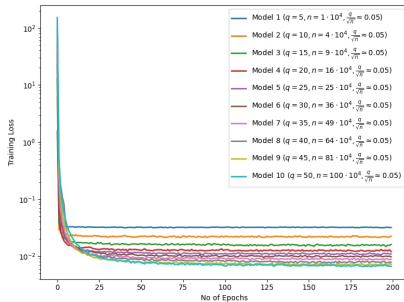


Figure 2: Training Loss vs Epoch in fixed $\frac{q}{\sqrt{n}}$ setting

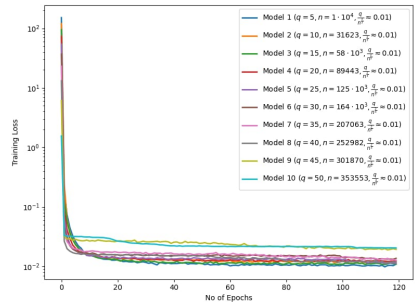


Figure 3: Training Loss vs Epoch in fixed $\frac{q}{n^{2/3}}$ setting

Future Work

Our theorem shows that a certain data size dependent largeness of q (the common output dimension) is needed if there has to exist a bounded weight DeepONet at that q which can have their empirical error below the label noise threshold.

Our experiments, have shown that there is some non-trivial range of q along which empirical risk improves with q for a fixed model size — if *the amount of training data is scaled quadratically with q .*

Q : Can we empirically check (for other PDEs) and prove such a “scaling law” ?

Future Work

This proof technique is modular (in terms of what are the main 4 lemmas required) and hence possibly provides a path to get similar lowerbounds for various ML setups. Lets try this on more models!

Future Work

This proof technique is modular (in terms of what are the main 4 lemmas required) and hence possibly provides a path to get similar lowerbounds for various ML setups. Lets try this on more models!

Physics Informed Neural Nets (PINNs) are an increasingly popular neural net setup which seems to solve various PDEs – but one PDE at a time, unlike DeepONets as we have been seeing.

We have run this technique on a PINN setup for solving d -variable Hamilton-Jacobi-Bellman PDEs and derived lowerbounds on the size of depth-2 nets required to lower the training error.

Future Work

This is a recently submitted work with my PhD student Sebastien Andre-Sloan and Prof. Matthew Colbrook @ DAMTP, Cambridge.

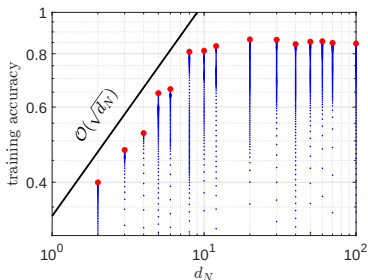


Figure 4: The vertical axis is $(1 - (\text{final training error}))$. The red dots represent the final training accuracy, while the blue dots show the training accuracy every 100 updates. The accuracy appears to grow like $\mathcal{O}(\sqrt{d_N})$ before plateauing at a level above one minus the variance.

Future Work

Informal Theorem Statement (with Sebastien Andre-Sloan and Matthew Colbrook)

Consider the PINN loss function for the HJB PDE where the first layer of the net is being trained. Let σ^2 be the average variance in the supervision data. If a predictor achieves an empirical risk $\mathcal{O}(\eta)$ below σ^2 , then necessarily,

$$d_N \log d_N \gtrsim N_s \eta^2$$

where N_s is the number of samples and d_N is the number of trainable parameters of the PINN.

Future Work

Informal Theorem Statement (with Sebastien Andre-Sloan and Matthew Colbrook)

Consider the PINN loss function for the HJB PDE where the first layer of the net is being trained. Let σ^2 be the average variance in the supervision data. If a predictor achieves an empirical risk $\mathcal{O}(\eta)$ below σ^2 , then necessarily,

$$d_N \log d_N \gtrsim N_s \eta^2$$

where N_s is the number of samples and d_N is the number of trainable parameters of the PINN.

Q : Is the above inequality true for a large class of PDEs?
(maybe..)

Questions?

